

SinghArpit2492 Chess Project

Chess Engine — Implementation Plan

Chess Engine — C# (.NET Console App)

Inspired by Sebastian Lague's "[Coding Adventure: Chess](#)" ([YouTube](#)) and his [GitHub repo](#).

Project Structure

```
d:\Chess\  
■■■ Chess.sln                                # Solution file  
■■■ ChessEngine\  
■   ■■■ ChessEngine.csproj                  # Project file  
■   ■■■ Program.cs                          # Entry point – game loop & UI  
■   ■  
■   ■■■ Core\  
■   ■   ■■■ Board.cs                        # Board state (8x8 int array, side-to-move, castli  
■   ■   ■■■ Piece.cs                        # Piece type constants & color helpers  
■   ■   ■■■ Move.cs                          # Move struct (from, to, flags for promotion/castl  
■   ■   ■■■ FenUtility.cs                    # Load/save positions via FEN strings  
■   ■   ■■■ Coord.cs                         # Rank/file coordinate helper  
■   ■  
■   ■■■ MoveGeneration\  
■   ■   ■■■ MoveGenerator.cs                 # Legal move generation for all piece types  
■   ■   ■■■ PrecomputedData.cs               # Direction offsets, knight jumps, distance-to-edg  
■   ■  
■   ■■■ Search\  
■   ■   ■■■ Searcher.cs                     # Minimax + Alpha-Beta pruning search  
■   ■   ■■■ MoveOrdering.cs                  # Move ordering heuristics (captures first, MVV-LV  
■   ■   ■■■ TranspositionTable.cs           # Zobrist hashing + transposition table (cache)  
■   ■  
■   ■■■ Evaluation\  
■   ■   ■■■ Evaluator.cs                     # Position evaluation (material, piece-square tabl  
■   ■   ■■■ PieceSquareTables.cs            # Positional bonus/penalty tables per piece type  
■   ■  
■   ■■■ UI\  
■       ■■■ ConsoleUI.cs                    # Board rendering in console with Unicode chess pi
```

Proposed Changes

Core — Board Representation

[NEW] [Piece.cs](#)

- Constants for piece types: `None=0`, `Pawn=1`, `Knight=2`, `Bishop=3`, `Rook=4`, `Queen=5`, `King=6`
- Color flags: `White=8`, `Black=16`
- Helper methods: `IsWhite()`, `IsBlack()`, `PieceType()`, `IsSliding()`

[NEW] [Coord.cs](#)

- Simple struct for `(file, rank)` board coordinates
- Convert to/from square index (0–63), algebraic notation ("e4")

[NEW] [Move.cs](#)

- Compact `ushort` move encoding: 6 bits from-square, 6 bits to-square, 4 bits flags
- Flags: `None`, `EnPassant`, `Castle`, `PawnTwoForward`, `PromoteQueen`, `PromoteRook`, `PromoteBishop`, `PromoteKnight`
- Properties: `StartSquare`, `TargetSquare`, `MoveFlag`

[NEW] [Board.cs](#)

- `int[] Squares` — 64-element array; each cell stores piece type + color
- Game state: `WhiteToMove`, `CastlingRights` (4-bit flags), `EnPassantSquare`, `FiftyMoveCounter`, `PlyCount`
- `int[] KingSquare` — tracks king position for each color
- `MakeMove(Move)` — applies a move, updates all state, pushes to undo stack
- `UnmakeMove(Move)` — restores previous state from undo stack
- Piece lists for fast lookup

[NEW] [FenUtility.cs](#)

- `LoadFromFen(string fen)` → sets up `Board` from FEN string
 - `CurrentFen(Board board)` → exports current position as FEN
 - Default starting position: `rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1`
-

Move Generation

[NEW] [PrecomputedData.cs](#)

- Direction offsets for sliding pieces: `{-8, 8, -1, 1, -7, 7, -9, 9}` (N, S, W, E, NW, SE, NE, SW)
 - `NumSquaresToEdge[64][8]` — precomputed distances to each board edge
 - Knight move offsets: `{-17, -15, -10, -6, 6, 10, 15, 17}`
 - Pawn attack offsets per color
-

[NEW] [MoveGenerator.cs](#)

- `GenerateMoves(Board board)` → returns `List<Move>` of all **legal** moves
 - Sliding piece logic (bishop, rook, queen) using direction offsets + edge distances
 - Knight moves with bounds checking
 - Pawn moves: single push, double push, captures, en passant, promotion
 - King moves: normal + castling (check legality: no pieces between, not moving through check)
 - **Legality check**: after generating pseudo-legal moves, filter out those that leave king in check
 - Check and pin detection
-

Search — AI (Minimax + Alpha-Beta)

[NEW] [Searcher.cs](#)

- **Iterative deepening**: search depth 1, 2, 3, ... up to a time limit or max depth
- **Minimax with Alpha-Beta pruning**: the core search algorithm
- **Quiescence search**: extend search for captures to avoid the "horizon effect"
- Returns `bestMove` and `bestEval` (evaluation in centipawns)
- Search statistics: nodes searched, time elapsed

[NEW] [MoveOrdering.cs](#)

- Order moves to improve alpha-beta cutoffs:
 - **Captures first** (MVV-LVA: Most Valuable Victim – Least Valuable Attacker)
 - **Promotions** scored highly
 - **Quiet moves** last
- Better move ordering → more cutoffs → faster search → deeper search

[NEW] [TranspositionTable.cs](#)

- **Zobrist hashing**: random 64-bit keys for each (piece, square) combination
 - Hash table to cache previously evaluated positions
 - Store: hash, depth, evaluation, flag (exact/lower/upper), best move
 - Avoids re-evaluating the same position reached by different move orders
-

Evaluation Function

[NEW] [Evaluator.cs](#)

- **Material counting**: Pawn=100, Knight=320, Bishop=330, Rook=500, Queen=900
 - **Piece-square table bonuses**: add positional value based on where each piece sits
-

- **Perspective:** return evaluation relative to the side-to-move (positive = good for current player)
- Future improvements: pawn structure, king safety, mobility

[NEW] [PieceSquareTables.cs](#)

- Static `int[]` arrays (64 values each) for: Pawn, Knight, Bishop, Rook, Queen, King (middlegame), King (endgame)
- Based on well-known tables from the chess programming community
- Pawns: center pawns valued higher; knights: centralized; bishops: long diagonals; rooks: 7th rank; king: castled position in middlegame, centralized in endgame

Console UI

[NEW] [ConsoleUI.cs](#)

- Render board using Unicode chess symbols: `♔♖♗♘♙♚♛♜♝♞♟`
- Color the squares (dark/light) using console background colors
- Show legal moves highlighted when a piece is selected
- Display: move history, captured pieces, evaluation score
- Input: algebraic notation (e.g., `e2e4`, `e7e5`)

[NEW] [Program.cs](#)

- Game loop: Human vs AI, Human vs Human, or AI vs AI modes
- Parse user input → find matching legal move → make move → AI responds
- Display board after each move
- Detect and announce: check, checkmate, stalemate, draw by repetition, 50-move rule

Key Algorithms (from the video)

Concept	Description
Board as <code>int[]</code>	Each square stores piece+color as a single <code>int</code>
Move as <code>ushort</code>	Compact 16-bit encoding for efficiency
Make/Unmake	Apply and undo moves without copying the board
Pseudo-legal → Legal	Generate all candidate moves, then filter for legality
Minimax	Recursive tree search — maximize your score, minimize opponent's
Alpha-Beta Pruning	Skip branches that can't possibly improve the result
Quiescence Search	Extend tactical positions to avoid misevaluation

Concept	Description
Iterative Deepening	Search deeper each iteration; stop when time runs out
Move Ordering	Search likely-best moves first for more alpha-beta cutoffs
Transposition Table	Cache evaluated positions using Zobrist hashing
Piece-Square Tables	Positional bonuses per piece per square

Verification Plan

Automated Tests

- **FEN parsing:** Load known positions and verify board state
- **Perft testing:** Count nodes at depth N from starting position and compare against known values:
 - Depth 1: 20 nodes
 - Depth 2: 400 nodes
 - Depth 3: 8,902 nodes
 - Depth 4: 197,281 nodes
 - Depth 5: 4,865,609 nodes
- **Build:** `dotnet build` must succeed with no errors
- **Run:** `dotnet run` must launch the game loop

Manual Verification

- Play a game against the AI and verify correct move execution
- Test special moves: castling (both sides), en passant, pawn promotion
- Verify checkmate detection and game termination

Open Questions

■ **Important:** ***Game mode preference**:* Should the default be ****Human (White) vs AI (Black)****, or would you also like an ****AI vs AI**** demo mode to watch the engine play itself?

■ **Note:** The engine will start at search depth 4-5 (responds in ~1-2 seconds). This is strong enough to beat casual players. Depth can be increased for stronger play at the cost of thinking time.